PRADITA University

IF140303-Web Application Development

Session-07:

Phoenix Framework & PostgreSQL

PRU/SPMI/FR-BM-18/0222

Alfa Yohannis



Contents

PRADITA University

1. Introduction to Phoenix Framework
2. Understanding the MVC Pattern in Phoenix
   - Integrating All Components
3. Installing PostgreSQL
4. Installing Phoenix Framework
5. Routing in Phoenix
6. Creating Page and Queue Controllers
7. Rendering HTML Pages
8. Embedding Templates
9. Summary

Phoenix in a Nutshell

PRADITA University

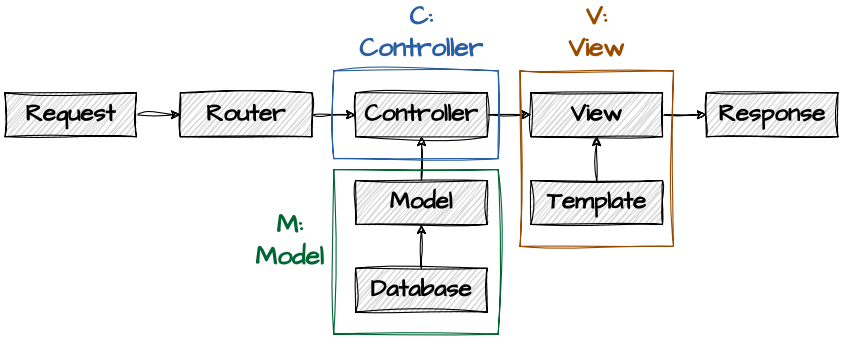
Phoenix is a full-stack Elixir web framework for scalable, real-time apps. Built on the Erlang VM (BEAM), it uses lightweight processes for massive concurrency and fault tolerance. It follows MVC, separating data, logic, and presentation.

- Fast request/response; real-time via Channels/WebSockets.
- Ecto for database access and composable queries.
- HEEx templates for safe, component-friendly HTML.

Ideal for chats, interactive dashboards, online games, and high-availability services of any size.

MVC Pattern in Phoenix

PRADITA University



Phoenix uses the Model-View-Controller (MVC) pattern for clear, reusable, and maintainable structure. **Model** handles data and logic via Ecto. **View** renders templates using HEEx. **Controller** manages requests and responses.

Request in Phoenix

PRADITA University

A request in Phoenix starts when a client (such as a web browser) sends an HTTP request to the server. The router matches this request to the appropriate controller and action for processing.

- The request contains a path, method, headers, and parameters.
- The controller receives the request via the `conn` struct.
- Phoenix passes the processed result to the view for rendering.

```
GET /about HTTP/1.1
Host: localhost:4000
User-Agent: Mozilla/5.0
Accept: text/html
```

This HTTP request is routed to `PageController.about/2`, which prepares data for rendering.

Router in Phoenix

PRADITA University

The router maps incoming HTTP requests to the appropriate controller and action. In Phoenix, routes are defined in the `router.ex` file. It checks the request path and method (GET, POST, etc.) to decide which controller and function should handle it.

```
scope "/", HelloWeb do
  pipe_through :browser

  get "/", PageController, :home
  get "/about", PageController, :about
  get "/queue/new", QueueController, :new
end
```

This example maps GET requests like `/about` and `/queue/new` to their respective controller actions.

Controller in Phoenix

PRADITA University

- Controllers act as a bridge between models (data) and views (presentation).
- They process incoming requests and may interact with the database.
- Each action corresponds to a function handling a specific request.
- Uses `render/3` to return the proper HTML template.
- The `conn` argument holds request and response data.

```elixir
defmodule HelloWeb.PageController do
  use HelloWeb, :controller

  def about(conn, _params) do
    render(conn, :about, layout: false)
  end

  def home(conn, _params) do
    render(conn, :home, layout: false)
  end
end
```

Model and Database Interaction

PRADITA University

Models represent and manage application data stored in a database. Phoenix uses `Ecto`, a robust database wrapper and query generator, for data operations.

- Defines schemas mapping Elixir structs to database tables.
- Supports CRUD operations (Create, Read, Update, Delete).
- Ensures data validation and integrity through changesets.

```elixir
defmodule Hello.Accounts.User do
  use Ecto.Schema

  schema "users" do
    field :name, :string
    field :email, :string
    field :age, :integer

    timestamps()
  end
end
```

View and Template in Phoenix

PRADITA University

Views are responsible for rendering responses such as HTML or JSON. They receive data from controllers and apply templates to generate the final output.

- Use `HEEx` (HTML Embedded Elixir) for dynamic HTML rendering.
- Contain no business logic — only presentation logic.
- Render controller data into client-ready formats.

```
<h1>
  <b>
    Hello World!
  </b>
</h1>
```

A simple HEEx template example for the `about` page.

Response in Phoenix

PRADITA University

After the view processes data and applies the template, Phoenix generates a final response and sends it back to the client.

- Responses can be HTML, JSON, or other content types.
- Determined by the request and controller action.
- The client (e.g., browser) renders the returned content.

```
GET /about
--> Returns:
about.html.heex

<h1><b>Hello World!</b></h1>
```

Example of an HTML response for the `/about` page.

Integrating All Components

PRADITA University

To summarize the MVC flow in Phoenix:

1. `Router` receives the HTTP request and forwards it to the appropriate `Controller`.
2. `Controller` handles business logic and may interact with the `Model` to fetch or update data.
3. `View` takes data from the `Controller` and uses a `Template` to render the response.
4. `Response` is sent back to the client, completing the request/response cycle.

By dividing the application into these components, Phoenix ensures each part has a single responsibility, improving maintainability and scalability over time.

Installing PostgreSQL (Part 1)

PRADITA University

PostgreSQL is a powerful open-source object-relational database system. It will serve as the database for our Phoenix web application. Follow these steps to install PostgreSQL:

1. Update package list:

```
sudo apt update
```

2. Install PostgreSQL and remove old configurations:

```
sudo apt install postgresql -y --purge
```

3. Enable and start the PostgreSQL service:

```
sudo systemctl enable postgresql
sudo systemctl start postgresql
sudo systemctl status postgresql
```

Installing PostgreSQL (Part 2)

PRADITA University

4. (Optional) Install pgAdmin4 for graphical management:

```
sudo apt install pgadmin4-desktop -y --purge
```

5. Switch to the PostgreSQL user:

```
sudo -i -u postgres
```

6. Set a password for the `postgres` user (for development, e.g. `1234`):

```
psql> \password postgres
```

After setup, PostgreSQL is ready for use with Phoenix.

Installing Phoenix Framework (Part 1)

ADITA iversity

Phoenix Framework, built on Elixir, enables scalable, real-time web development. Follow these steps to install Phoenix and set up a new project:

1. Install `inotify-tools` to detect file changes.

```
sudo apt install inotify-tools
```

2. Check Elixir version.

```
elixir -v
```

3. Install Hex, the package manager for Elixir.

```
mix local.hex
```

4. Install the Phoenix project generator.

```
mix archive.install hex phx_new
```

5. Create a new project named `hello`.

```
mix phx.new hello
```

Installing Phoenix Framework (Part 2)

ADITA iversity

1. Enter the project directory and open in VS Code.

```
cd hello
code .
```

2. Edit database settings in `config/dev.exs` and `config/test.exs`, set password to `1234`.
3. Fetch dependencies and create the database.

```
mix deps.get
mix ecto.create
```

3. Start the Phoenix server.

```
mix phx.server
iex -S mix phx.server
```

4. Visit the app in your browser:

```
http://localhost:4000
```

Phoenix is now installed and running!

Routing in Phoenix

PRADITA University

Routing in Phoenix defines how incoming requests are mapped to specific controllers and actions. Follow these steps to configure routes in your application:

1. Open `lib/hello_web/router.ex` and define the following routes:

```
scope "/", HelloWeb do
  pipe_through :browser

  get "/", PageController, :home
  get "/about", PageController, :
      about
  get "/queue/new", QueueController,
       :new
end
```

1. `pipe_through :browser`: Applies the default browser stack, including sessions, cookies, and request handling.
2. `get "/"`: Maps the root URL to the `home` action in `PageController`.
3. Similarly, `get "/about"` and `get "/queue/new"` define routes for the About page and New Queue page.

Creating Controllers (Part 1)

PRADITA University

Controllers in Phoenix handle logic and rendering for different web pages. Let's define two controllers: one for general pages and another for queue actions.

1. Define `PageController` to handle the home and about pages.
2. `use HelloWeb, :controller`: Imports Phoenix controller functionality.
3. `about/2`: Renders `about.html.heex` without layout.
4. `home/2`: Renders `home.html.heex`.

```
1  defmodule HelloWeb.PageController
       do
2    use HelloWeb, :controller
3
4    def about(conn, _params) do
5      render(conn, :about, layout:
           false)
6    end
7
8    def home(conn, _params) do
9      render(conn, :home, layout:
           false)
10   end
11 end
```

Creating Controllers (Part 2)

PRADITA University

3. Define `QueueController` to handle the new queue page:

```elixir
defmodule HelloWeb.QueueController do
  use HelloWeb, :controller

  def new(conn, _params) do
    render(conn, :new, layout: false)
  end
end
```

4. `new/2`: Renders `new.html.heex` for the queue creation page.

Rendering HTML Templates in Phoenix

HTML templates define the structure of web pages rendered by controllers. Let's create two templates — one for the About page and one for the New Queue page.

About Template:

```
<h1>
  <b>
    Hello World!
  </b>
</h1>
```

This simple HTML template displays a "Hello World" message for the About page.

New Queue Template:

```
<h1>
  <b>
    This is the New Queue Page
  </b>
</h1>
```

This template displays a simple heading for the New Queue page. Templates like these are rendered through controller actions using `render/3`.

Embedding Templates in Phoenix

PRADITA University

```elixir
defmodule HelloWeb.QueueHTML do
  @moduledoc """
  This module contains the pages rendered
  by the QueueController.

  See the `queue_html` directory for all
  available templates.
  """
  use HelloWeb, :html

  embed_templates "queue_html/*"
end
```

Phoenix allows you to embed HTML templates directly inside modules, making it easier to manage multiple templates for a single controller. In this example, the `QueueHTML` module is used for `QueueController` templates.

- The `embed_templates` macro automatically imports all templates from the `queue_html` directory.
- This approach keeps controllers lightweight and simplifies the rendering workflow.

Summary

PRADITA University

- **PostgreSQL:** Installed and configured as the database for Phoenix apps.
- **Phoenix Setup:** Installed dependencies, created a new project, and ran the server.
- **MVC Pattern:** Organized the app with `Model`, `View`, and `Controller`.
- **Routing:** Mapped HTTP requests to controller actions.
- **Controllers:** Built `PageController` and `QueueController`.
- **Templates:** Rendered HTML views using HEEx templates.
- **Embedded Templates:** Used `embed_templates` for easy template management.
- **Result:** A working Phoenix web app connected to PostgreSQL.